

EVE310 Lab 5

October 9, 2025

Announcements

- **No graded Lab Quiz this week**
- Activity in Canvas (involves making some changes to the Lab 6 Tutorial code and observing how results are impacted)
- Answer sheet for activity posted to self-check your work.

Learning objectives

- Importing specific functions from a package
- Splitting data between training and testing datasets
- Creating linear regression models with Sklearn
- Analyzing and plotting model results
- Compute adjusted r-squared
- Use one-hot encoding to include categorical variables

This lab is a direct extension of what you learned in Lecture 10 and the traffic example code you ran.

Importing Functions from a Package

- Sometimes we want to import an entire package (like NumPy)

```
import numpy as np
```

- Sometimes, we only want to use a specific function from a package.
We can import only that function using:

```
from <package> import <function>
```

- In practice, it looks like this:

```
from sklearn.linear_model import LinearRegression
```

- When we use the LinearRegression function later, we can just use the function as is. Does not require anything before the function (compared to using the np.sum function where the np needs to be specified)

Importing Packages (and Functions from Packages)

```
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Fitting a simple linear model using sklearn
6 from sklearn.linear_model import LinearRegression
7 from sklearn.model_selection import train_test_split
8 from sklearn.metrics import r2_score
9 from sklearn.preprocessing import OneHotEncoder
10
11 %matplotlib inline
```

Example

Austin department of transportation needs to assess if it should add another lane to Mopac. To plan whether to add additional lane, we need to estimate the expected traffic volume in a typical day. The first step towards that is to estimate the number of trips taken by a typical household in Austin. You've conducted a survey polling the number of people in a household and the number of average trips taken each day.

# people	1	2	3	4	5	1	2	3	4	5	1	2	3	4	5
# trips	0.9	1.8	2.4	4.3	6.6	1.9	2.7	3.6	5.4	7.7	2.1	3.1	3.7	5.9	8.3

- Find a **model** that **estimates** the number of trips as a function of size of the household.

Import Data and reshape if necessary

```
1 # create an array
2 npeople = np.array([1,2,3,4,5,1,2,3,4,5,1,2,3,4,5])
3 ntrips = np.array([0.9,1.8,2.4,4.3,6.6,1.9,2.7,3.6,5.4,7.7,2.1,3.1,3.7,5.9,8.3])
4 #reshape arrays
5 x = npeople.reshape(-1, 1)
6 y = ntrips.reshape(-1,1)
```

- If using data from a csv, here is where you would import it to a DataFrame

Split Data between Training and Testing sets

Specify a random seed for reproducibility (ensures same split every time). Note: this does not always need to be done. Only when it makes sense in your context.

```
#Split between training and testing  
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=45)
```

70% of x data

30% of x data

70% of
corresponding y data

30% of
corresponding y data

Full x dataset

Full y dataset

Proportion of
data to be used in
testing set

Split Data between Training and Testing sets

Specify a random seed for reproducibility (ensures same split every time). Note: this does not always need to be done. Only when it makes sense in your context.

```
#Split between training and testing
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=45)
```

70% of x data

30% of x data

70% of
corresponding y data

30% of
corresponding y data

Full x dataset

Full y dataset

Proportion of
data to be used in
testing set

# People	1	2	3	4	5	1	2	3	4	5	1	2	3	4	5
# Trips	0.9	1.8	2.4	4.3	6.6	1.9	2.7	3.6	5.4	7.7	2.1	3.1	3.7	5.9	8.3

Training
Testing

Create Model, Fit to Training Data, and Predict

```
#Initiate model  
lm = LinearRegression()  
  
#Fit model  
lm.fit(X_train,y_train)
```

Note: Later in the course, we will be using other types of models. The same process applies, but instead of using `LinearRegression()`, it will be a different type of model.

```
#Predict y values from training dataset  
y_train_predicted = lm.predict(X_train)
```

```
#Predict y values from testing dataset  
y_test_predicted = lm.predict(X_test)
```

Quantify Model Error

Predict values from
training, test datasets

R^2 using `r2_score`
function that we
previously imported

Adjusted R^2 , see
lecture slides for
formula

```
#Predict y values from training dataset
y_train_predicted = lm.predict(X_train)

#Compute r squared from training dataset
r2_train = r2_score(y_train,y_train_predicted)

#Compute adjusted rsquared from training dataset
n = X_train.shape[0]
p = X_train.shape[1]

r2_train_adj = 1-(((1-r2_train)*(n-1))/(n-p-1))

#-----

#Predict y values from testing dataset
y_test_predicted = lm.predict(X_test)

#Compute r squared from testing dataset
r2_test = r2_score(y_test,y_test_predicted)

#Compute adjusted rsquared from testing dataset
n = X_test.shape[0]
p = X_test.shape[1]

r2_test_adj = 1-(((1-r2_test)*(n-1))/(n-p-1))
```

Adjusted R-Squared

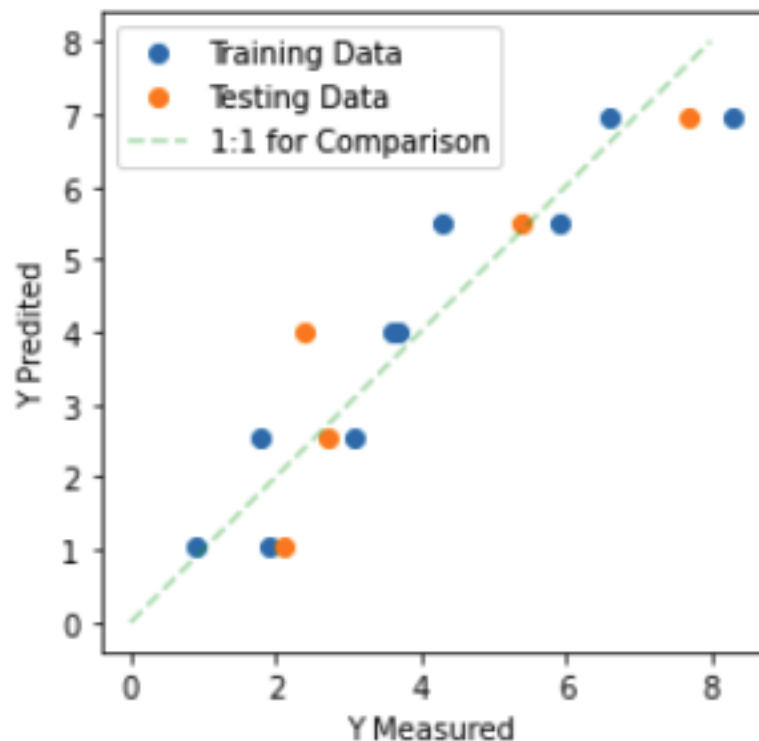
- Adjusted R^2 (see lecture slides for formula)
- Remember the .shape function and indexing
- $n = 12, p = 1$

```
print(X_train.shape)
```

```
(12, 1)
```

```
#Compute adjusted rsquared from training dataset  
n = X_train.shape[0]  
p = X_train.shape[1]  
  
r2_train_adj = 1 - (((1 - r2_train) * (n - 1)) / (n - p - 1))
```

Quantify Model Error



#Plot Y Measured against Y Predicted

```
f=plt.figure()
```

```
p1=plt.plot(y_train,y_train_predicted,marker='o',ls='')
```

```
p2=plt.plot(y_test,y_test_predicted,marker='o',ls='')
```

#The alpha option makes the line slightly lighter and more transparent

```
p3=plt.plot(np.array([0,8]),np.array([0,8]),ls='--',alpha=.35)
```

```
ax=plt.gca()
```

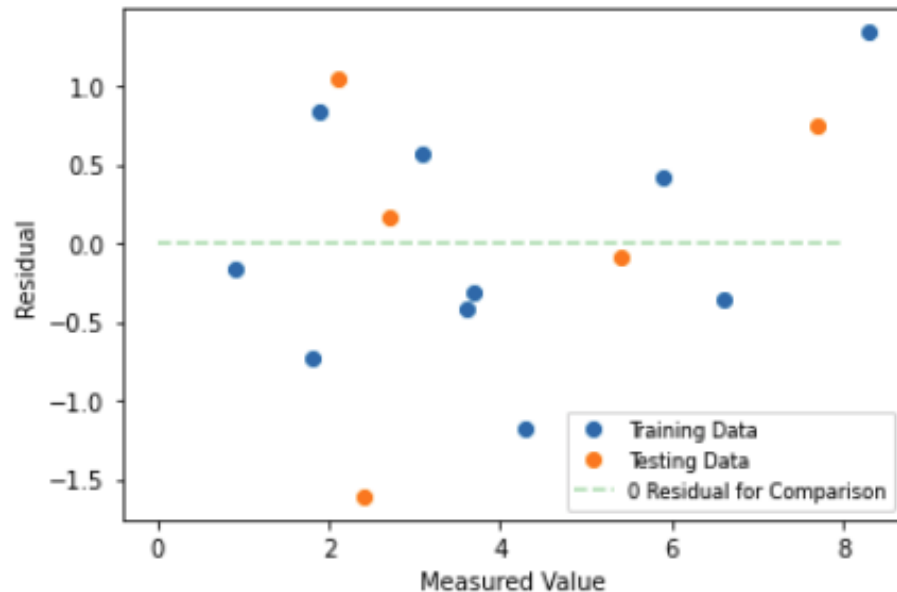
```
ax.set_aspect('equal')
```

```
ax.legend(['Training Data','Testing Data','1:1 for Comparison'])
```

```
plt.ylabel('Y Predicted')
```

```
plt.xlabel('Y Measured')
```

Quantify Model Error



#Calculate and plot residuals

```
residual_train=y_train-y_train_predicted
```

```
residual_test=y_test-y_test_predicted
```

#Create the figure

```
plt.figure()
```

```
p1=plt.plot(y_train,residual_train,marker='o',ls='')
```

```
p2=plt.plot(y_test,residual_test,marker='o',ls='')
```

```
p3=plt.plot(np.array([0,8]),np.array([0,0]),ls='--',alpha=.35)
```

```
ax=plt.gca()
```

```
ax.legend(['Training Data','Testing Data','0 Residual for Comparison'],loc=4,fontsize='small')
```

```
plt.ylabel('Residual')
```

```
plt.xlabel('Measured Value')
```

Creating a New Model to Include Categorical Variables

Step 1: Input the categorical array

Add categorical covariates:

# people	1	2	3	4	5	1	2	3	4	5	1	2	3	4	5
# work	No	Part-time	Part-time	Full-time	Full-time	Part-time	Part-time	Part-time	Full-time	Full-time	No	Full-time	Full-time	Full-time	Full-time
# trips	0.9	1.8	2.4	4.3	6.6	1.9	2.7	3.6	5.4	7.7	2.1	3.1	3.7	5.9	8.3

```
#Adding covariate- categorical variable  
#no=1  
#part-time=2  
#full-time=3  
  
#Create array from data given  
nwork=np.array([1,2,2,3,3,2,2,2,3,3,1,3,3,3,3])  
  
#reshape array  
nwork = nwork.reshape(-1, 1)
```


Dummy coding

- **Example:**
- Science = 1 & Math = 0, then Language = 0
- Science = 0 and Math = 1, then Language = 0
- Science = 0 and Math = 0, then Language = 1

Student	Favorite class
1	Science
2	Math
3	Science
4	Language
5	Math

Student	Science	Math
1	1	0
2	0	1
3	1	0
4	0	0
5	0	1

Remove dependent variable
Recall: i.i.d.

- k categories
- $k - 1$ dummy binary (0,1) variables

Introduce Dummy Variables

Step 2: Use 1-Hot encoding to create dummy variables

```
#initiate encoder
one_hot = OneHotEncoder()

#1-hot encode categorical variables
nwork_cat = one_hot.fit_transform(nwork)

#Convert to numpy array
cat_array = nwork_cat.toarray()

#The hot_array is printed to see what it looks like
print(cat_array)
```

We only need 2 of the 3 columns



$$\begin{bmatrix} [1. & 0. & 0.] \\ [0. & 1. & 0.] \\ [0. & 1. & 0.] \\ [0. & 0. & 1.] \\ [0. & 0. & 1.] \\ [0. & 1. & 0.] \\ [0. & 1. & 0.] \\ [0. & 1. & 0.] \\ [0. & 0. & 1.] \\ [0. & 0. & 1.] \\ [1. & 0. & 0.] \\ [0. & 0. & 1.] \\ [0. & 0. & 1.] \\ [0. & 0. & 1.] \\ [0. & 0. & 1.] \end{bmatrix}$$

Add categorical covariates:

# people	1	2	3	4	5	1	2	3	4	5	1	2	3	4	5
# work	No	Part-time	Part-time	Full-time	Full-time	Part-time	Part-time	Part-time	Full-time	Full-time	No	Full-time	Full-time	Full-time	Full-time
# trips	0.9	1.8	2.4	4.3	6.6	1.9	2.7	3.6	5.4	7.7	2.1	3.1	3.7	5.9	8.3

Concatenate Dummy Variables with Continuous Variables

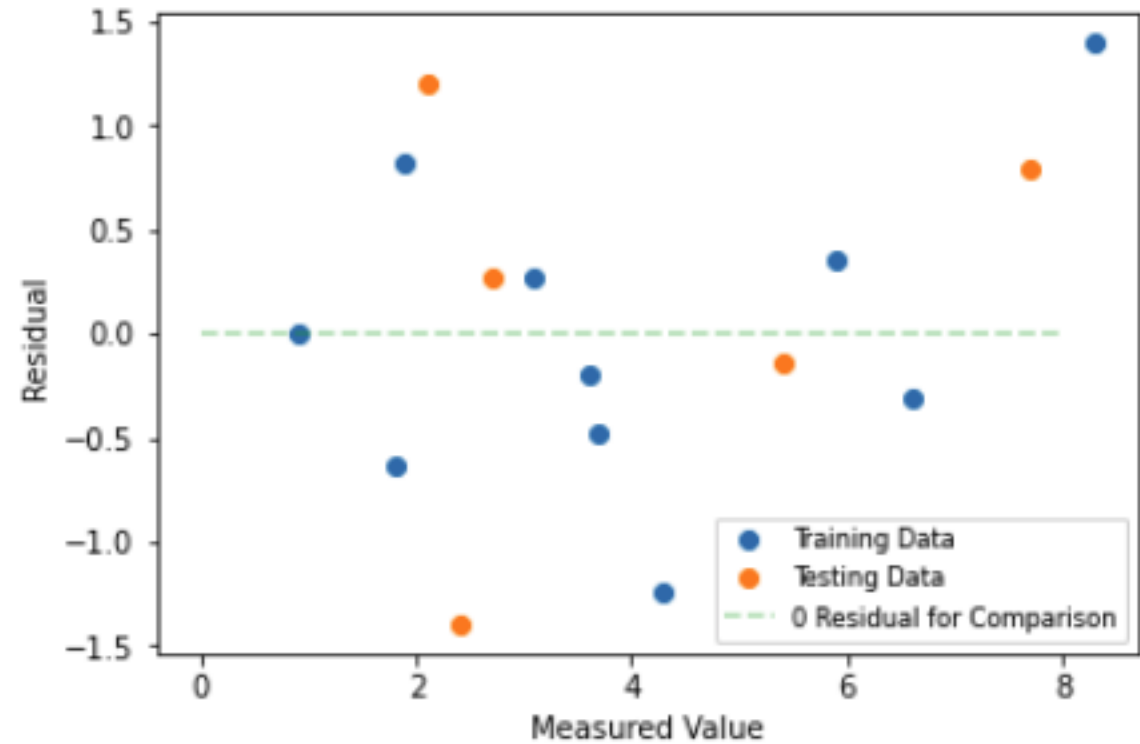
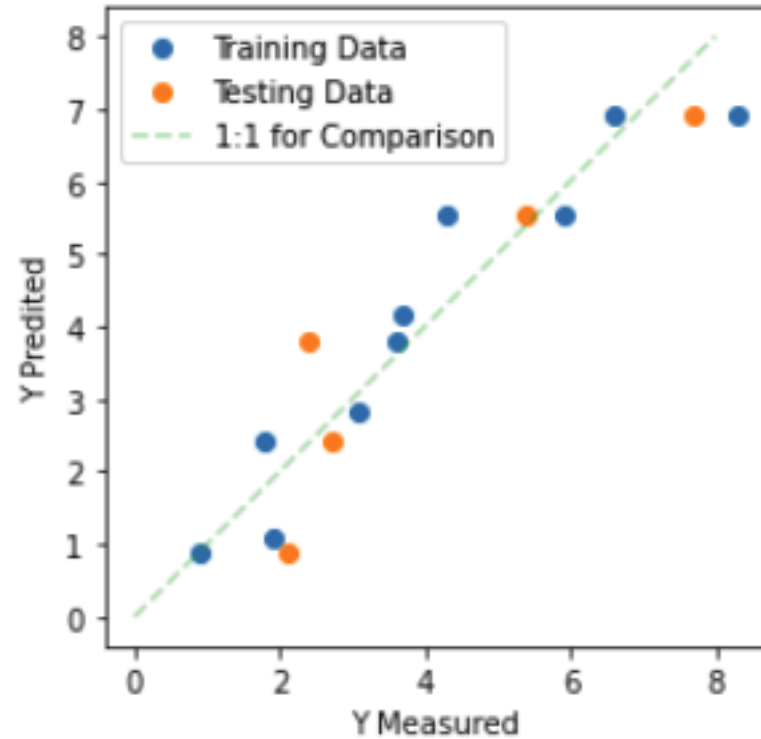
```
#Next, concatenate this new array to existing x data  
x_with_cat = np.hstack((x, cat_array[:,0:2]))  
  
#Print the concatenated array - this is our new x data  
print(x_with_cat)
```

Use only the first and
second columns of
cat_array, not all 3
(see previous slide)

```
[[1. 1. 0.]  
 [2. 0. 1.]  
 [3. 0. 1.]  
 [4. 0. 0.]  
 [5. 0. 0.]  
 [1. 0. 1.]  
 [2. 0. 1.]  
 [3. 0. 1.]  
 [4. 0. 0.]  
 [5. 0. 0.]  
 [1. 1. 0.]  
 [2. 0. 0.]  
 [3. 0. 0.]  
 [4. 0. 0.]  
 [5. 0. 0.]]
```

Here is an example of using indexing that we learned earlier in the class!

Repeat all previous steps with x-data which includes dummy variables for categorical variables



Compare R Squared Values

```
print('The rsquared value of the training data with 1 covariate was',r2_train)
print('The rsquared value of the training data with 3 covariates was',r2_train_cat)

print('The adjusted rsquared value of the training data with 1 covariate was',r2_train_adj)
print('The adjusted rsquared value of the training data with 3 covariates was',r2_train_adj_cat)

print('-----')

print('The rsquared value of the testing data with 1 covariate was',r2_test)
print('The rsquared value of the testing data with 3 covariates was',r2_test_cat)

print('The adjusted rsquared value of the testing data with 1 covariate was',r2_test_adj)
print('The adjusted rsquared value of the testing data with 3 covariates was',r2_test_adj_cat)
```

```
The rsquared value of the training data with 1 covariate was 0.8903906361906321
The rsquared value of the training data with 3 covariates was 0.8946612629245868
The adjusted rsquared value of the training data with 1 covariate was 0.876689465714461
The adjusted rsquared value of the training data with 3 covariates was 0.8419918943868803
-----
The rsquared value of the testing data with 1 covariate was 0.8188425847096884
The rsquared value of the testing data with 3 covariates was 0.8251285720475297
The adjusted rsquared value of the testing data with 1 covariate was 0.7584567796129179
The adjusted rsquared value of the testing data with 3 covariates was 0.3005142881901186
```

Activity

Recap

- Today's lab was about the mechanics of making a model and interpreting results - next week will be an extension of this.
- Next week, we will learn how to implement scaling, outlier removal, and interaction terms.
- Review these slides, the tutorial, activity, and activity answer key for next week.